RAG-Based Document Q&A Application

Complete Project Plan & Technical Blueprint

Portfolio Project for Career Switch | Version 1.0

Engineer	Aviguhan (Associate Software Engineer, 2.2 yrs exp)
Tech Stack	React + MUI + TypeScript	ASP.NET Core 8	Semantic Kernel	Qdrant
Target Role	Full-Stack / AI-integrated Developer
Timeline	4 Weeks (Part-time, ~2 hrs/day)
Date	21 March 2026

1. Executive Summary

This document outlines the complete project plan for building a RAG (Retrieval-Augmented Generation) based Document Q&A Application as a portfolio project. The application allows users to upload PDF documents and ask natural language questions, receiving AI-generated answers grounded in the document content.

This project is specifically designed to showcase:

- Full-stack engineering across React (frontend) and ASP.NET Core (backend)
- Practical AI integration using OpenAI embeddings and LLM via Microsoft Semantic Kernel
- Modern DevOps practices including Docker, CI/CD, and cloud deployment
- Production-grade code quality with authentication, error handling, and clean architecture

Upon completion, this project alone can position you for roles targeting 15-25 LPA, particularly at product companies building AI-powered tools.

2. Project Overview

2.1 Problem Statement

Professionals and students frequently need to extract insights from large documents — legal contracts, research papers, technical manuals, and reports. Reading through hundreds of pages is time-consuming. This app solves that by enabling natural language Q&A directly over uploaded documents.

2.2 Solution

A web application where users upload PDF documents, which are processed through a RAG pipeline: the documents are chunked, embedded into vectors, and stored in a vector database. When a user asks a question, the most relevant chunks are retrieved and passed to an LLM to generate a grounded, accurate answer with source attribution.

2.3 Key Features

- **PDF document upload with drag-and-drop interface**
  - Support for multi-page PDFs up to 50MB
  - Upload progress indicator with validation
- **Natural language Q&A over uploaded documents**
  - Real-time streaming responses with typewriter effect
  - Source chunk highlighting showing which part of the document answered
- **Multi-document collection management**
  - Create named collections (e.g., 'Legal Docs', 'Research Papers')
  - Query across multiple documents in a collection
- **Conversation history with context awareness**
  - Multi-turn conversations maintaining context
  - Chat history persisted per user session
- **User authentication and authorization**
  - JWT-based auth with refresh tokens
  - Each user's documents are private and isolated
- **Responsive UI with React MUI**
  - Dark/light mode toggle
  - Mobile-friendly layout

3. System Architecture

3.1 High-Level Architecture

The application follows a clean 3-tier architecture with a separate AI processing pipeline:

Layer	Technology	Responsibility
Presentation	React 18 + MUI + TypeScript	UI, state management, streaming display
API Gateway	ASP.NET Core 8 Web API	REST endpoints, auth, request routing
AI Pipeline	Semantic Kernel + OpenAI	Embedding, retrieval, LLM orchestration
Vector Store	Qdrant (Docker)	Semantic similarity search over embeddings
Relational DB	PostgreSQL + EF Core	User data, document metadata, chat history
Infrastructure	Docker Compose	Container orchestration for local dev

3.2 RAG Pipeline Flow

The RAG pipeline has two distinct phases:

Ingestion Phase (when document is uploaded):

- User uploads PDF via React frontend
- ASP.NET Core API receives file and saves metadata to PostgreSQL
- PdfPig library extracts raw text from the PDF
- Text is split into overlapping chunks (500 tokens, 50-token overlap)
- Each chunk is sent to OpenAI Embeddings API (text-embedding-ada-002)
- Embedding vectors + chunk text stored in Qdrant vector database

Query Phase (when user asks a question):

- User types a question in the React chat interface
- Question is embedded using the same OpenAI embeddings model
- Qdrant performs cosine similarity search, returning top 5 most relevant chunks
- Chunks are assembled into a context block with source references
- A structured prompt is built: Context + Question + Instructions
- Prompt is sent to OpenAI GPT-4o, response streamed back via SSE
- Frontend renders the answer in real-time with source citations shown below

4. Detailed Technology Stack

4.1 Frontend

Package	Version	Purpose
React	18.x	UI framework with concurrent rendering
TypeScript	5.x	Type safety across entire frontend codebase
MUI (Material UI)	5.x	Component library for professional UI
Vite	5.x	Build tool — fast HMR for development
Axios	1.x	HTTP client for API calls
React Router DOM	6.x	Client-side routing (login, dashboard, chat)
React Dropzone	14.x	Drag-and-drop file upload component
React Markdown	9.x	Render LLM markdown responses beautifully
Zustand	4.x	Lightweight global state management
React Query	5.x	Server state, caching, and background refetch

4.2 Backend

Package	Version	Purpose
ASP.NET Core	8.0	Web API framework
Microsoft Semantic Kernel	1.x	AI orchestration — embeddings, memory, LLM
Semantic Kernel Qdrant Connector	1.x	Connect Semantic Kernel to Qdrant vector DB
PdfPig	0.1.8	PDF text extraction (open source, no license issues)
EF Core	8.x	ORM for PostgreSQL
Npgsql EF Core Provider	8.x	PostgreSQL driver for EF Core
JwtBearer Authentication	8.x	JWT validation middleware
BCrypt.Net-Next	4.x	Password hashing
Serilog	3.x	Structured logging to console and file
FluentValidation	11.x	Request validation with clean rules

4.3 Infrastructure

Tool	Purpose	Notes
Docker Desktop	Containerize Qdrant + PostgreSQL	Single docker-compose.yml manages both
Qdrant	Vector database	Runs on localhost:6333, free & open source
PostgreSQL 16	Relational data	Runs on localhost:5432 via Docker
OpenAI API	Embeddings + LLM	text-embedding-ada-002 + GPT-4o
GitHub	Version control	Public repo boosts your profile visibility

5. Project Folder Structure

5.1 Backend — ASP.NET Core

DocQnA.API/

- Controllers/ — AuthController.cs, DocumentController.cs, QnAController.cs
- Services/ — DocumentService.cs, EmbeddingService.cs, QnAService.cs, AuthService.cs
- Models/ — User.cs, Document.cs, Chunk.cs, ChatMessage.cs (EF Core entities)
- DTOs/ — Request and Response data transfer objects
- Infrastructure/ — AppDbContext.cs, Migrations/, QdrantConfig.cs
- Middleware/ — ExceptionHandlingMiddleware.cs, JwtMiddleware.cs
- appsettings.json — App config (OpenAI endpoint, DB strings)
- appsettings.Development.json — Secrets (API keys) — gitignored!
- Program.cs — DI registrations, middleware pipeline, Semantic Kernel setup

5.2 Frontend — React + TypeScript

doc-qna-client/src/

- pages/ — LoginPage.tsx, DashboardPage.tsx, ChatPage.tsx
- components/ — DocumentUploader.tsx, ChatWindow.tsx, SourceViewer.tsx, CollectionSelector.tsx, Sidebar.tsx
- api/ — auth.ts, documents.ts, qna.ts (Axios wrappers)
- hooks/ — useAuth.ts, useDocuments.ts, useChat.ts
- store/ — authStore.ts, chatStore.ts (Zustand stores)
- types/ — index.ts (TypeScript interfaces for all entities)
- theme/ — theme.ts (MUI custom theme with dark mode)
- utils/ — streamParser.ts, tokenHelpers.ts

6. Week-by-Week Development Plan

Estimated time: 2 hours per day, 5 days per week. Total: ~40 hours over 4 weeks.

Week 1 — Backend Core & Document Ingestion

Day	Task	Output
Day 1	Project setup — create ASP.NET solution, install packages, setup docker-compose.yml for Qdrant + Postgres, configure EF Core connection	Runnable empty API, DB connected
Day 2	Auth system — User model, JWT login/register endpoints, BCrypt password hashing, JWT middleware	POST /auth/register and /auth/login working
Day 3	Document upload endpoint — receive IFormFile, validate type/size, save metadata to Postgres via EF Core	POST /documents/upload saves file info to DB
Day 4	PDF text extraction + chunking — integrate PdfPig, implement sliding window chunker with configurable size and overlap	ExtractAndChunk() method tested on sample PDF
Day 5	Embedding + Qdrant storage — integrate Semantic Kernel, call OpenAI embeddings, store vectors in Qdrant collection	Full ingestion pipeline end-to-end working

Week 2 — Q&A API & Streaming

Day	Task	Output
Day 1	Q&A service — embed question, search Qdrant for top-5 chunks, assemble context + prompt	SearchRelevantChunksAsync() tested via Postman
Day 2	LLM integration — send prompt to GPT-4o via Semantic Kernel, get answer, return with source chunk metadata	POST /qna/ask returns JSON answer + sources
Day 3	Streaming endpoint — refactor to use Server-Sent Events (SSE), stream token-by-token from OpenAI streaming API	GET /qna/ask-stream streams response live
Day 4	Chat history — persist Q&A pairs per user to Postgres, GET /qna/history endpoint	Chat history stored and retrievable
Day 5	Collection management — CRUD for document collections, associate documents to collections, query across collection	GET/POST/DELETE /collections endpoints done

Week 3 — React Frontend

Day	Task	Output
Day 1	Project setup — Vite + React + TS, MUI theme, routing with React Router, Zustand auth store, Axios interceptors for JWT	App shell with routing and auth state
Day 2	Auth pages — LoginPage and RegisterPage with MUI forms, form validation, connect to auth API, redirect on success	Login/Register UI fully working
Day 3	Dashboard + Upload — DashboardPage with CollectionSelector, DocumentUploader with drag-and-drop, upload progress bar, file list	User can upload PDFs and see them listed
Day 4	Chat UI — ChatWindow with message bubbles, question input, send button, connect to streaming API, render tokens live with typewriter effect	Real-time Q&A working in the browser
Day 5	Source viewer + polish — SourceViewer panel showing retrieved chunks, markdown rendering of answers, dark/light mode toggle, mobile responsiveness	App looks professional and polished

Week 4 — Polish, Deploy & Portfolio Prep

Day	Task	Output
Day 1	Error handling — global exception middleware in backend, error boundaries in React, toast notifications for all errors, loading skeletons	App handles all edge cases gracefully
Day 2	Testing — unit tests for DocumentService and QnAService (xUnit), React component tests (Vitest + Testing Library) for core components	Test suite with >60% coverage
Day 3	README + docs — write detailed GitHub README with screenshots, architecture diagram, setup guide, and tech decisions explained	Professional README that impresses recruiters
Day 4	Deployment — Dockerize the ASP.NET API, deploy frontend to Vercel (free), deploy API to Railway or Render (free tier)	Live demo URL to put on resume
Day 5	Portfolio write-up — LinkedIn post about the project, update resume bullet points, record a 2-minute demo video for GitHub README	Resume updated, project publicly visible

7. API Endpoint Reference

7.1 Authentication

Method	Endpoint	Description	Auth Required
POST	/api/auth/register	Register new user with email + password	No
POST	/api/auth/login	Login, returns JWT access token + refresh token	No
POST	/api/auth/refresh	Refresh expired access token	No
DELETE	/api/auth/logout	Invalidate refresh token	Yes

7.2 Documents

Method	Endpoint	Description	Auth Required
POST	/api/documents/upload	Upload PDF, triggers ingestion pipeline	Yes
GET	/api/documents	List all documents for current user	Yes
DELETE	/api/documents/{id}	Delete document and its vectors from Qdrant	Yes
GET	/api/documents/{id}/status	Check ingestion status (processing/ready/failed)	Yes

7.3 Q&A

Method	Endpoint	Description	Auth Required
POST	/api/qna/ask	Ask a question, returns full answer + sources (JSON)	Yes
GET	/api/qna/ask-stream	Ask a question with SSE streaming (token-by-token)	Yes
GET	/api/qna/history	Get conversation history for current user	Yes
DELETE	/api/qna/history	Clear all conversation history	Yes

7.4 Collections

Method	Endpoint	Description	Auth Required
GET	/api/collections	List all user collections	Yes
POST	/api/collections	Create a new named collection	Yes
POST	/api/collections/{id}/documents	Add document to collection	Yes
DELETE	/api/collections/{id}	Delete collection and optionally its documents	Yes

8. Database Schema

PostgreSQL stores all relational data. Qdrant stores all vector embeddings separately.

8.1 Users Table

Column	Type	Notes
Id	UUID (PK)	Primary key
Email	VARCHAR(255) UNIQUE	Used for login
PasswordHash	VARCHAR(255)	BCrypt hashed password
CreatedAt	TIMESTAMP	Registration timestamp
RefreshToken	VARCHAR(500)	Current valid refresh token
RefreshTokenExpiry	TIMESTAMP	Token expiry for refresh validation

8.2 Documents Table

Column	Type	Notes
Id	UUID (PK)	Primary key
UserId	UUID (FK)	References Users.Id
OriginalFileName	VARCHAR(255)	Display name
QdrantCollectionName	VARCHAR(255)	The Qdrant collection where vectors are stored
ChunkCount	INT	Number of chunks created during ingestion
Status	VARCHAR(50)	processing / ready / failed
CreatedAt	TIMESTAMP	Upload timestamp
FileSizeBytes	BIGINT	For storage tracking

8.3 ChatMessages Table

Column	Type	Notes
Id	UUID (PK)	Primary key
UserId	UUID (FK)	References Users.Id
DocumentId	UUID (FK, nullable)	The document queried against

Column	Type	Notes
Question	TEXT	User's question
Answer	TEXT	LLM-generated answer
SourceChunks	JSONB	Array of retrieved chunk texts + scores
CreatedAt	TIMESTAMP	When the Q&A happened

9. How to Present This on Your Resume

9.1 Resume Bullet Points

Under a 'Projects' section, add the following:

RAG-Based Document Q&A Platform | React, MUI, TypeScript, ASP.NET Core 8, Semantic Kernel, Qdrant, OpenAI

- Architected and built a full-stack AI application enabling natural language Q&A over user-uploaded PDF documents using Retrieval-Augmented Generation (RAG)
- Implemented document ingestion pipeline with text extraction (PdfPig), semantic chunking, and vector embedding storage in Qdrant via Microsoft Semantic Kernel
- Designed real-time streaming Q&A using Server-Sent Events (SSE) with source attribution, rendering LLM responses token-by-token in a React MUI chat interface
- Built JWT-based auth system with refresh tokens, multi-collection document management, and persistent chat history using PostgreSQL + EF Core
- Containerized backend infrastructure with Docker Compose; deployed frontend to Vercel and API to Railway with live demo at [your-url-here]

9.2 Key Talking Points for Interviews

- Why RAG over fine-tuning? — Explain cost, freshness of data, and transparency of sources
- Chunking strategy decisions — Why 500 tokens with 50-token overlap? Trade-offs with chunk size
- Streaming implementation — How SSE works vs WebSockets, why you chose SSE here
- Vector search vs full-text search — When cosine similarity wins over LIKE queries
- Semantic Kernel vs raw OpenAI SDK — Abstraction benefits and the plugin/memory model

9.3 GitHub Repository Checklist

- Descriptive README.md with architecture diagram, screenshots, and setup guide
- Live demo link (Vercel + Railway free tier)
- Proper .gitignore — never commit API keys or appsettings.Development.json
- Clean commit history with meaningful messages (one feature per commit)
- LICENSE file (MIT recommended for portfolio projects)
- GitHub Actions CI workflow to run tests on every push

10. Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
OpenAI API costs exceed budget	Medium	High	Set hard usage limits in OpenAI dashboard; use GPT-3.5-turbo during development, GPT-4o only for final demo
Qdrant version incompatibility with Semantic Kernel connector	Low	Medium	Pin Docker image version in docker-compose.yml; check Semantic Kernel release notes
PDF extraction fails on complex layouts	Medium	Low	Use PdfPig with fallback to raw text extraction; note limitation in README
Streaming SSE breaks with CORS	Medium	Medium	Configure CORS policy in ASP.NET Core to allow EventSource headers
Scope creep extends timeline	High	Medium	Stick to MVP features in Weeks 1-3; add enhancements only in Week 4 if time permits

11. Estimated Costs

This project can be built and demonstrated for under $10 USD total.

Item	Cost	Notes
OpenAI API (development)	~$3-5 USD	Embeddings are cheap; most cost is GPT-4o queries during testing
OpenAI API (demo)	~$1-2 USD	Live demo with limited usage
Qdrant	Free	Open source, self-hosted via Docker
PostgreSQL	Free	Self-hosted via Docker
Vercel (frontend hosting)	Free	Hobby tier is sufficient
Railway (API hosting)	Free tier	$5 credit/month on free plan — sufficient for demos
GitHub	Free	Public repository
Total Estimated	< $10 USD	Approximately Rs. 850 one-time

This document is your north star. Build it week by week, ship it, and own the conversation in every interview.

Good luck, AviGupta!